



Факултет по Математика и Информатика
ПУ "Паисий Хилендарски"

ПАРОЛИ, ХЕШИРАНЕ

Витали Атиас

Защо паролите са уязвими – човешкият фактор

В повечето уеб системи паролата е първата и често единствена линия на защита на идентичността. Това означава, че сигурността на акаунта зависи от:

- тайността на паролата
- сложността ѝ
- начина на съхранение в системата

Проблемът с човешкото поведение

Хората:

- използват лесни пароли (123456, password, admin)
- използват една и съща парола в различни сайтове (password reuse)
- записват пароли в небезопасни места
- избират кратки и предвидими комбинации

Типични атаки срещу пароли

Типични атаки срещу пароли

- Brute force – изпробване на всички възможни комбинации
- Dictionary attack – използване на списък с популярни пароли
- Credential stuffing – използване на изтекли пароли от други системи
- Phishing – измама за доброволно разкриване на паролата

Извод:

Дори при силна инфраструктура, слабата парола компрометира сигурността на системата.

Съхранение на пароли – защо `plain text` е критична грешка

Най-простият начин за съхраняване на пароли е да се запишат директно в базата данни:

Това е изключително опасно.

Ако базата данни бъде компрометирана, нападателят получава всички пароли. Това означава, че той може:

- да влезе в акаунтите
- да използва същите пароли в други сайтове

Hash функции – основа на сигурното съхранение

Вместо да съхранява паролата директно, системата съхранява hash на паролата.

Hash функция е математическа функция, която преобразува входни данни в фиксирана стойност.

Пример:

password → hash

Например:

mypassword → 34819d7beeabb9260a5c854bc85b3e44

Hash функции – основа на сигурното съхранение

Основни свойства

1. One-way (еднопосочност) - не може да се възстанови оригиналният вход.
2. Deterministic - един и същ вход → един и същ hash.
3. Fixed length - изходът има фиксирана дължина.

Как работи authentication с hash

При регистрация:

Потребителят въвежда парола:

mypassword

Системата изчислява hash:

hash(mypassword) =

34819d7beeabb9260a5c854bc85b3e44

Системата съхранява hash, не паролата:

username: student1

hash: 34819d7beeabb9260a5c854bc85b3e44

При login:

Потребителят въвежда:

mypassword

Системата изчислява hash:

hash(mypassword)

Сравнява със съхранения hash.

Ако съвпадат → login успешен.

MD5 – Message Digest Algorithm 5

MD5 е криптографска hash функция, разработена през 1991 година. Тя генерира 128-bit hash.

Пример:

MD5("password") =

5f4dcc3b5aa765d61d8327deb882cf99

MD5 е бил широко използван, защото е:

- бърз
- лесен за използване
- широко поддържан

Dictionary attack

Dictionary атака използва списък с често използвани пароли.

Пример dictionary:

password

123456

admin

Нападателят изчислява MD5 hash за всяка парола.

Rainbow tables

Rainbow table е предварително изчислена таблица.

Пример:

password → 5f4dcc3b5aa765d61d8327deb882cf99

123456 → e10adc3949ba59abbe56e057f20f883e

Ако нападателят има hash:

5f4dcc3b5aa765d61d8327deb882cf99

той просто проверява таблицата.

Не е нужно да изчислява hash.

Collision проблем при MD5

Collision означава, че две различни стойности имат един и същ hash.

Пример:

input1 → hash

input2 → hash

MD5 е доказано уязвим на collision атаки.

Това означава, че нападател може да създаде различни входове със същия hash.

Това прави MD5 неподходящ за сигурност.

Salt – защита срещу rainbow table attack

*Salt прави rainbow table атаки
практически невъзможни, защото
всеки hash е уникален.*

Salt е случайна стойност, която се добавя към
паролата.

Пример:

password = mypassword

salt = random123

Hash:

hash(mypassword + random123)

Системата съхранява: hash & salt

Защо MD5 не трябва да се използва за пароли

Основните причини:

MD5 е бърз → лесен brute force

MD5 има collisions

MD5 няма защита срещу GPU атаки

MD5 е компрометиран криптографски

Основни принципи за сигурност на пароли

Никога не съхранявайте plain text

Никога не използвайте MD5

Винаги използвайте salt

Използвайте специализирани password hashing
алгоритми